How Well Does a 0.5B Model Follow Instructions?

Evaluating Qwen2.5-0.5B-Instruct on IFEval, and reading the gap to the paper

Class project · Module 3: Evaluation · September 2026

At a glance

Model	Qwen2.5-0.5B-Instruct (commit `7ae5576`)
Benchmark	IFEval: 541 prompts, 834 verifiable instructions
Scorer	Official Google scorer, unmodified (commit `ff4d1a6`)
Headline	**26.6%** of prompts fully followed (strict), 95% CI 23.1–30.5%
vs. model developer	Qwen report 27.9%, **inside our interval**
vs. IFEval paper	GPT-4 76.9%, PaLM 2 S 43.1%
Harness check	Re-scoring the paper's GPT-4 responses reproduces its strict numbers **exactly**
Repeatability	4 identical runs produced **byte-identical** generations
System prompt	3 variants moved the headline by −0.2 to +2.0 points, **none significant**
Cost	18.6 min on a laptop RTX 3050 Ti (0.31 GPU-hours)

---

1. What we set out to do

IFEval measures whether a model follows instructions that a program can check, such as "*answer in all lowercase*", "*use no commas*" or "*write exactly 3 paragraphs*". Because every instruction is checked by code, not by a human or another model, the score is objective and repeatable.

We built an **evaluation harness** that a stranger can re-run with one command, used it to measure a small open-weight model, and compared the result with published numbers.

> **Important framing:** the IFEval paper (Zhou et al., 2023) evaluated GPT-4 and PaLM 2 S, not Qwen. So this is **not a reproduction** of a paper number. It is a measurement of a different, much smaller model on the same instrument. The one thing we *can* reproduce exactly is the scorer, and we did (Section 3).

---

2. Method

Pipeline

```
IFEval prompts (541)
  → build_prompts.py   apply the chat template, save the exact string sent
  → generate.py        model writes one response per prompt       [GPU]
  → score.py           official scorer → pass/fail per instruction
  → analyze.py         4 metrics + 95% CI, per-type, length, truncation
```

One command runs everything and writes a new timestamped folder under `runs/`. Nothing is ever overwritten.

The four metrics

Metric	A prompt / instruction counts as passed when…
Prompt-level strict	*every* instruction in the prompt passes, checked on the raw response
Instruction-level strict	each instruction is counted separately
Prompt-level loose	as strict, but the scorer also tries lightly cleaned versions of the response (e.g. markdown `*` removed, first/last line dropped)
Instruction-level loose	loose, counted per instruction

Settings used

Setting	Value	Why it matters
Model revision	`7ae557604adf…` (pinned)	"main" can change under us
Precision / attention	fp16 / SDPA	target hardware (V100) has no bf16 or FlashAttention-2
Decoding	greedy, `max_new_tokens = 1024`	deterministic baseline
Hidden decoding defaults	`repetition_penalty = 1.1`, `top_k = 20`	come from the model's own `generation_config.json` (only the penalty affects greedy)
Prompt format	model's chat template, empty system prompt	template inserts Qwen's default system message (see below)
Batch size	4	batch 8 ran out of memory on the 4 GB laptop GPU
Hardware	NVIDIA RTX 3050 Ti Laptop, 4 GB	specified V100 was unavailable (Section 8)
Software	torch 2.4.1+cu121, transformers 4.44.2, Python 3.10.14	all pinned in `requirements.txt`

The exact string the model received for the first prompt:

```
<|im_start|>system
You are Qwen, created by Alibaba Cloud. You are a helpful assistant.<|im_end|>
<|im_start|>user
Write a 300+ word summary of the wikipedia page "…Raymond_III,_Count_of_Tripoli".
Do not use any commas and highlight at least 3 sections …<|im_end|>
<|im_start|>assistant
```

---

3. Validating the instrument before trusting it

A score is only meaningful if the measuring tool is correct. We ran three checks.

Check 1: The scorer reproduces the paper

The scorer repository ships the GPT-4 responses used in the paper. We scored them with our pinned scorer:

GPT-4	Prompt strict	Instr. strict	Prompt loose	Instr. loose
Paper (Table 3)	76.89	83.57	79.30	85.37
Our scorer (7 scorings)	**76.89**–77.08	**83.57**–83.69	79.48–79.67	85.49–85.61

- **Strict: exact match** in 6 of 7 scorings.
- **Loose: 0.2–0.4 points higher** than the paper in every scoring. This is an unexplained residual. Likely candidates are changes to the scorer's loose mode since 2023 and the prompt edit described below. It is too small to affect any conclusion here.

Check 2: Two issues found in the official benchmark (documented, not patched)

1. **The scorer is not fully deterministic.** Two prompts (keys 1122 and 1129) ask how often the characters `#` and `!` appear. The `letter_frequency` checker rejects non-letters and silently substitutes a **random letter** each time it runs. The same responses can therefore score differently. *Measured impact on our run (8 re-scorings):* prompt-level unchanged, instruction-level varies by **0.12 points**.
2. **The benchmark file changed after the paper.** Prompt 2785 now asks for "at least **3** placeholders"; the paper's GPT-4 answers were written for "at least **one** placeholder". The paper's response file no longer scores without adjustment (we remapped that one prompt for Check 1).

Check 3: The pipeline runs end-to-end without a model

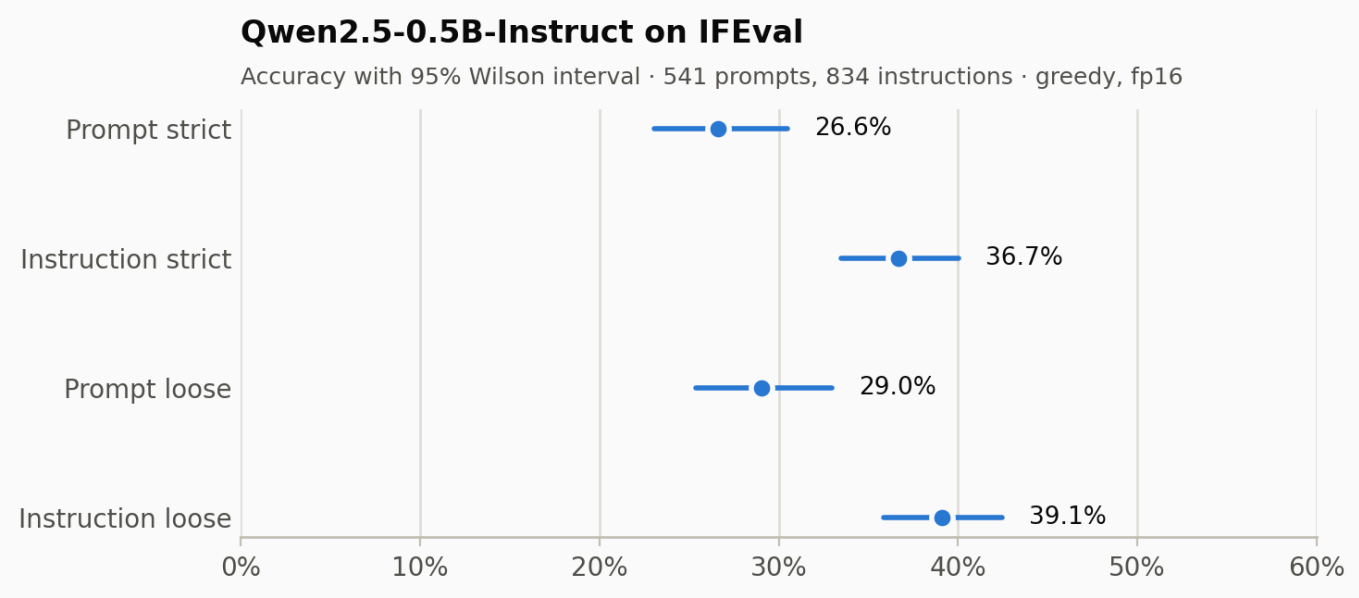
A "dummy" mode that returns canned text runs the full pipeline on any laptop in about 3 seconds and scores above zero, which proves the scorer is wired correctly before spending GPU time.

---

4. Results

Headline metrics

Metric	Accuracy	95% CI (Wilson)	n
Prompt-level strict	**26.6%**	23.1 – 30.5%	541 prompts
Instruction-level strict	36.7%	33.5 – 40.0%	834 instructions
Prompt-level loose	29.0%	25.4 – 33.0%	541 prompts
Instruction-level loose	39.1%	35.8 – 42.4%	834 instructions

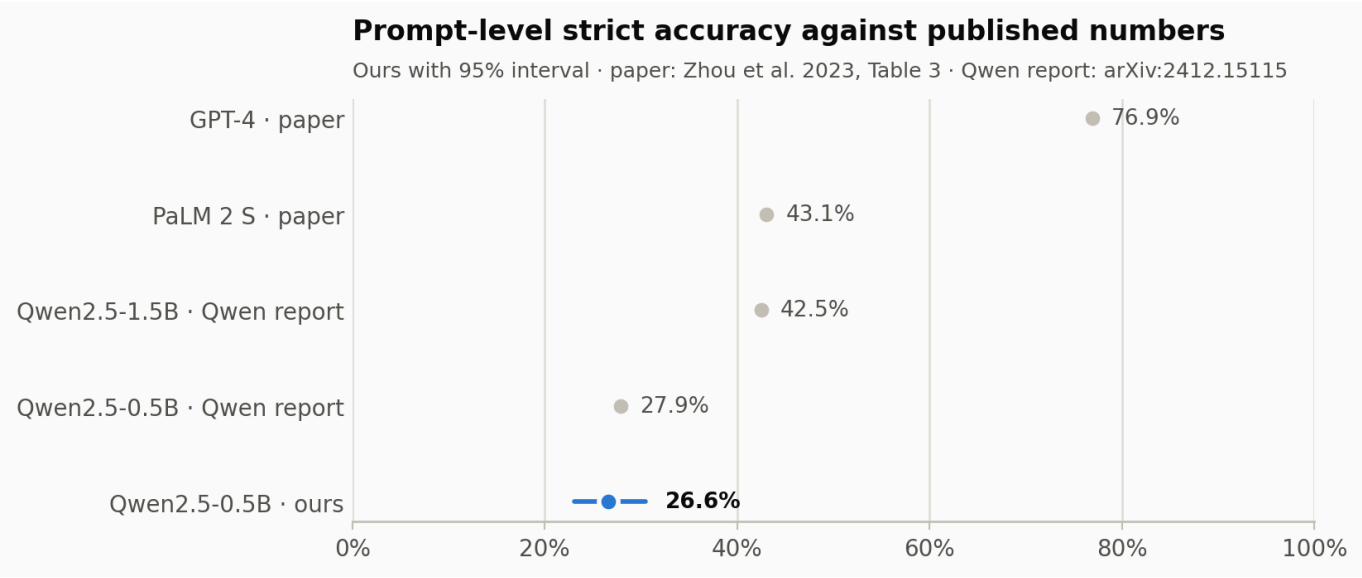


Run-to-run spread: the baseline was run 4 times with identical settings. All 4 runs produced byte-identical generations, so prompt-level strict, prompt-level loose and instruction-level loose were identical in every run. Instruction-level strict ranged from 36.6% to 36.7% (0.12 points). That variation comes only from the scorer's random letter check (Section 3), not from the model.

Comparison with published numbers (prompt-level strict)

Model	Size	Prompt-level strict	Source
GPT-4	undisclosed	76.9%	IFEval paper, Table 3
PaLM 2 S	undisclosed	43.1%	IFEval paper, Table 3
Qwen2.5-1.5B-Instruct	1.5B	42.5%	Qwen2.5 Technical Report*
Qwen2.5-0.5B-Instruct	0.5B	27.9%	Qwen2.5 Technical Report*
Qwen2.5-0.5B-Instruct (ours)	**0.5B**	**26.6%** (23.1–30.5)	this harness

* The report lists a single IFEval figure; 27.9 lies inside our confidence interval for both prompt-level metrics, so the comparison holds either way.

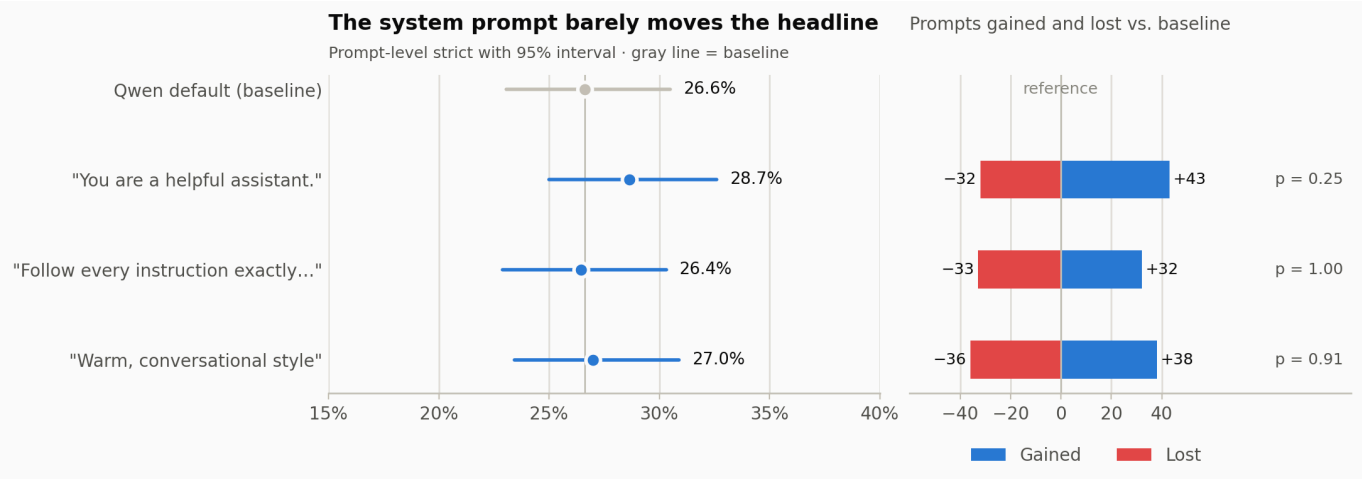


Gaps: −50.3 points to GPT-4, −16.5 points to PaLM 2 S, −1.3 points to Qwen's own report.

Sensitivity to the system prompt

With an empty system prompt, Qwen's chat template inserts its own default ("*You are Qwen, created by Alibaba Cloud. You are a helpful assistant.*"). We replaced it with three alternatives and changed nothing else.

System prompt	Prompt strict	Instr. strict	Prompt loose	Instr. loose	Truncated	Prompts gained / lost	p
Qwen default (baseline)	26.6%	36.7%	29.0%	39.1%	3.7%	—	—
"You are a helpful assistant."	28.7%	39.9%	31.8%	43.6%	6.1%	+43 / −32	0.25
"…Follow every instruction … exactly, including all formatting, length, casing and punctuation requirements."	26.4%	37.4%	29.6%	41.2%	2.8%	+32 / −33	1.00
"…writes in a warm, natural, conversational style."	27.0%	36.8%	29.6%	39.6%	3.7%	+38 / −36	0.91



Gained / lost: prompts that failed in the baseline and passed with the variant, and the reverse (prompt-level strict). p: exact McNemar test on those paired prompts.

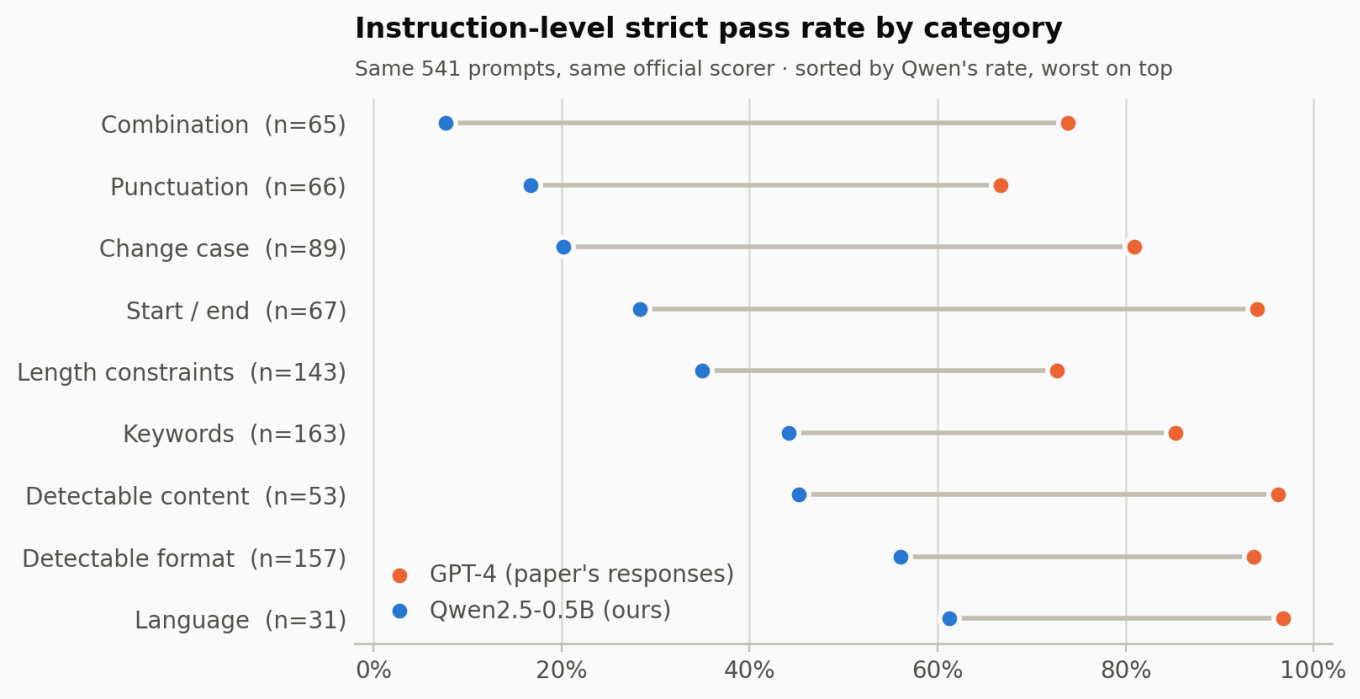
- **Changing the system prompt rewrote almost every response.** Only 9–18 of 541 responses stayed identical to the baseline, and 65–75 prompts changed outcome.
- **The headline barely moved.** Gains and losses nearly cancel: the net effect on prompt-level strict ranges from −0.2 to +2.0 points, and no variant differs significantly from the baseline.
- **Telling the model to follow instructions did not help** (−0.2 points).
- **The generic prompt gave the largest, non-significant gain** (+2.0 points). It came mostly from change case (+12.4 points) and detectable content (+11.3 points), and it also raised truncation to 6.1%.

---

5. Where the model fails

By instruction category (instruction-level strict)

Category	Qwen 0.5B	GPT-4	n
Language (`response_language`)	61.3%	96.8%	31
Detectable format (JSON, title, bullets…)	56.1%	93.6%	157
Detectable content (postscript, placeholders)	45.3%	96.2%	53
Keywords (include / avoid / frequency)	44.2%	84.7%	163
Length constraints (words, sentences, paragraphs)	35.0%	72.7%	143
Start / end (quotation, end phrase)	28.4%	94.0%	67
Change case (all caps / all lowercase)	20.2%	80.9%	89
Punctuation (no commas)	16.7%	66.7%	66
Combination (repeat prompt, two responses)	7.7%	73.8%	65



GPT-4 column: paper's published responses, scored by our harness.

The five hardest and five easiest instruction types

Hardest	Pass	Easiest	Pass
`number_paragraphs`	0 / 27	`title`	34 / 37
`repeat_prompt`	3 / 41	`constrained_response`	9 / 10
`two_responses`	2 / 24	`json_format`	12 / 17
`nth_paragraph_first_word`	1 / 12	`postscript`	17 / 26
`english_capital`	3 / 25	`response_language`	19 / 31

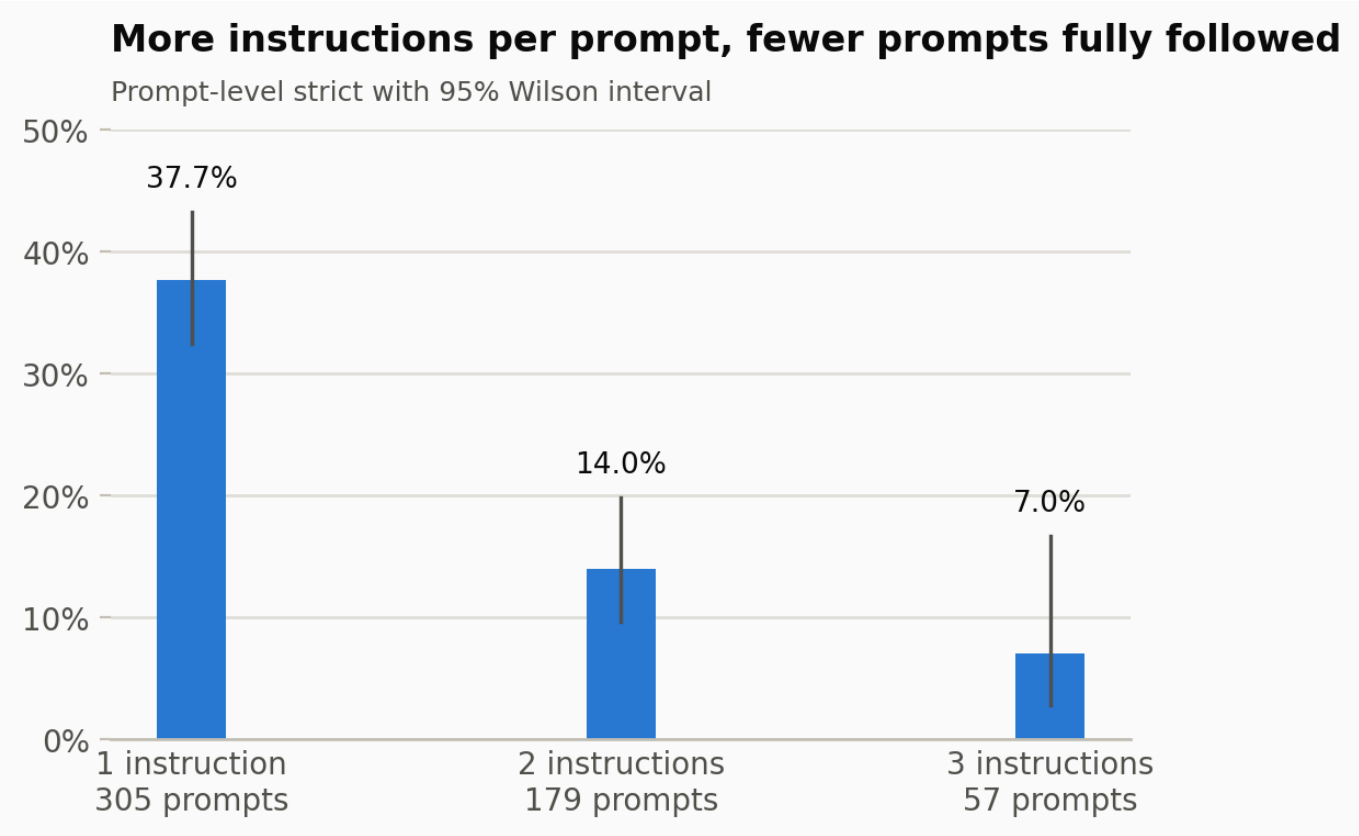
What the failures look like

Instruction	What was asked	What the model did
Paragraphs (0%)	"Separate the paragraphs with the markdown divider `***`"	Used blank lines instead: only 4 of 27 responses contained `***` at all
Two responses (8%)	"Give two responses separated by `******`"	Wrote a numbered list `1. … 2. …`
Repeat prompt (7%)	"First repeat the request word for word, then answer"	Skipped the repetition and answered directly
All lowercase (15%)	"Write a letter … in all lowercase letters"	Opened with `Dear [Friend's Name],`, using normal capitalisation
No commas (17%)	"Do not use any commas"	Used commas from the first line

Pattern: the model does the *task* (writes the letter, the story, the email) but ignores the *formatting constraint* attached to it. Constraints that match common writing habits (titles, JSON, postscripts) pass; constraints that fight habits (no capitals, no commas, unusual separators) fail.

More instructions → much lower success

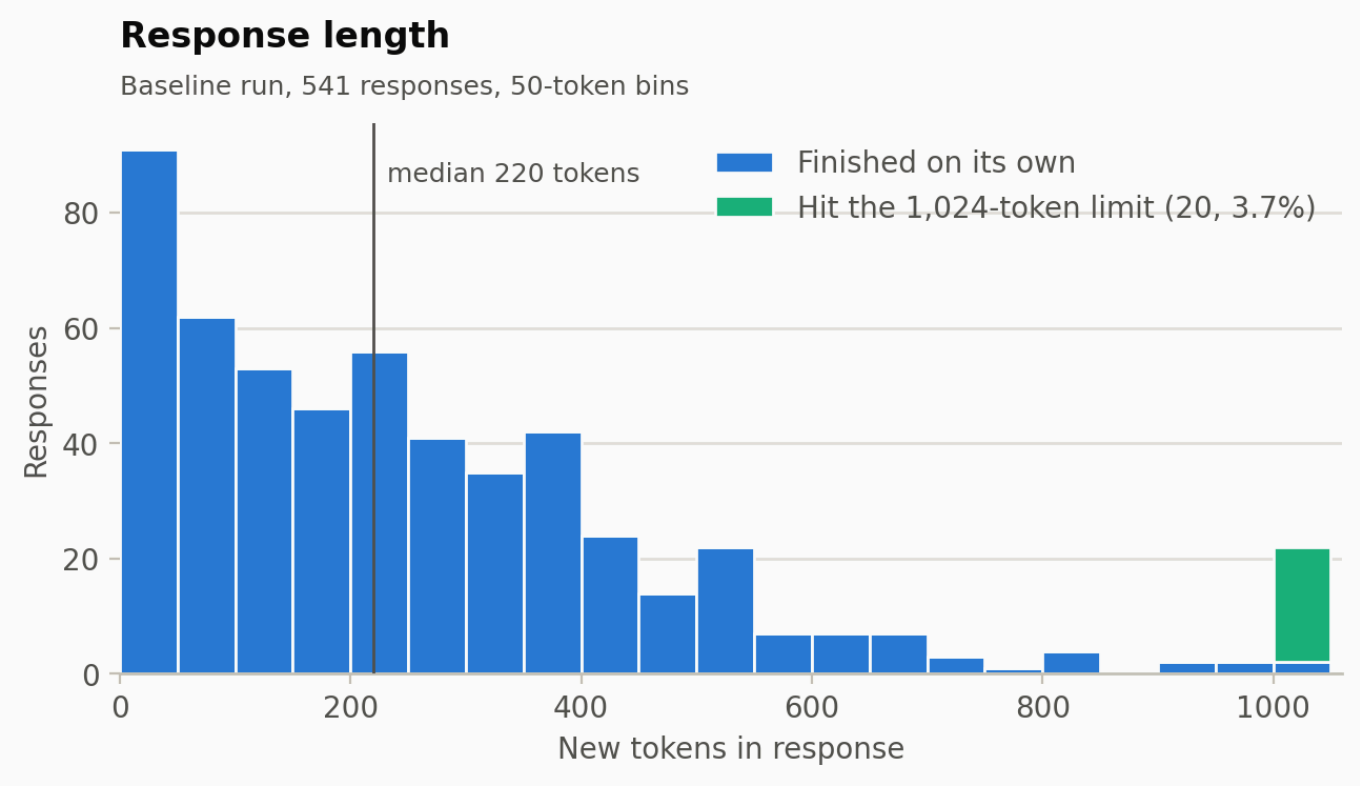
Instructions in the prompt	Prompts fully followed
1	115 / 305 = **37.7%**
2	25 / 179 = **14.0%**
3	4 / 57 = **7.0%**



This is why prompt-level accuracy (26.6%) sits well below instruction-level (36.7%).

6. Response length and truncation

Median length	220 tokens
90th percentile	550 tokens
Hit the 1024-token limit	**20 responses (3.7%)**
Truncated responses that still passed	8 of 20
Truncated responses stuck in a repetition loop	~8 of 20 (last lines repeat)



Upper bound on the truncation effect: even if all 12 failing truncated responses had passed, prompt-level strict would rise by at most **2.2 points** (to 28.8%). Truncation does not explain the gap to PaLM 2 S or GPT-4.

---

7. Reading the gap

Against Qwen's own report (−1.3 points): no meaningful gap. Their 27.9% lies inside our 95% interval (±3.7 points). Our harness agrees with the model developer's measurement. Differences in system prompt, decoding defaults or hardware could each move the number, but none is needed to explain a 1.3-point difference.

Against PaLM 2 S (−16.5) and GPT-4 (−50.3): a real capability gap, dominated by scale. These are different, far larger models, so the gap is not a harness error: the same scorer reproduces GPT-4's strict numbers exactly. Known confounds, and how large each can be:

Confound	Direction	Size (measured or bounded)
Model scale (0.5B vs. much larger models)	lowers ours	dominant, not separable in this study
Truncation at 1024 tokens	lowers ours	**≤ 2.2 points** (bounded above)
Scorer randomness (`letter_frequency` bug)	either	**≤ 0.12 points** instruction-level, 0 prompt-level
Benchmark edit (prompt 2785)	either	≤ 0.18 points (1 prompt in 541)
Default system prompt inserted by template	either	**−0.2 to +2.0 points** across 3 variants, none significant (Section 4)
Run-to-run variation (same hardware and settings)	none	**0**: 4 runs gave byte-identical generations (Section 4)

Confound	Direction	Size (measured or bounded)
`repetition_penalty = 1.1` from model defaults	unknown	not varied (Section 8)
Laptop GPU and batch 4 instead of specified V100 and batch 8	either, likely small	not measurable: hardware unavailable (Section 8)

Honest bottom line: the measurable, bounded confounds add up to under 5 points (truncation ≤ 2.2, system prompt ≤ 2.0, scorer and benchmark issues ≤ 0.3), far smaller than the 16- and 50-point gaps. What remains is best explained by model capability. That is consistent with the steep drop on prompts that stack several instructions, and with the model ignoring format constraints while still completing the task.

---

8. Hardware constraint and limitations

Hardware constraint

The project specifies an NVIDIA V100 (fp16, batch size 8). That hardware was unavailable for the duration of the project, so **all runs were performed on a 4 GB NVIDIA RTX 3050 Ti laptop GPU**. Model commit, software versions, precision (fp16), attention implementation (SDPA) and decoding settings are the same as specified. The only change is batch size 4 instead of 8, because batch 8 ran out of memory on the longest prompts.

What this does not affect

- **Scoring.** The scorer runs on the CPU and gives the same result on any machine. It reproduces the paper's GPT-4 numbers (Section 3).
- **What the model was given.** Weights, prompt strings and decoding settings are pinned and recorded in every run directory.
- **Comparisons within this report.** Every run uses the same GPU and batch size.
- **The conclusions in Section 7.** The gaps to PaLM 2 S and GPT-4 are 16 and 50 points.

What it can affect

- **Exact greedy outputs.** A different GPU architecture (Ampere instead of Volta) and different batch padding change fp16 arithmetic slightly. That can flip individual tokens, and occasionally an item's pass/fail. Its size relative to a V100 cannot be measured without a V100.
- **Absolute cost.** 18.6 minutes (0.31 GPU-hours) is a laptop figure and does not transfer to other hardware.

The Ampere GPU supports bf16 and FlashAttention-2, but neither was used, so the runs stay within the specified V100 settings.

Other limitations

- **Confidence intervals assume independence.** Instructions inside one prompt are correlated, so the instruction-level interval is somewhat too narrow.
- **Decoding defaults used as shipped.** `repetition_penalty = 1.1` from the model's own generation config applied to every run and was not varied.

- **First baseline run predates version control.** It records `git_commit: none`. The three repeats, run from committed code (`b8303f4`, `fbdd4b1`), reproduced its generations byte for byte.

Further work: saving generations after each batch would let an interrupted run resume; this was not implemented or measured here.

---

9. Reproducing this result

```
python3 -m venv .venv && .venv/bin/pip install -r requirements.txt
./scripts/setup.sh
PYTHON=.venv/bin/python ./run.sh configs/qwen05b_laptop.yaml hf    # 4 GB GPU
PYTHON=.venv/bin/python ./run.sh configs/qwen05b.yaml hf           # specified V100 config
(batch 8)
```

The system-prompt variants use `configs/qwen05b_laptop_sys_{generic,strict,casual}.yaml`.

Every chart above is regenerated from `runs/` by `report/evaluation_figures.ipynb` (`pip install -r requirements-notebook.txt`), which writes `report/figures/*.png`.

Every run behind this report has its own directory under `runs/`:

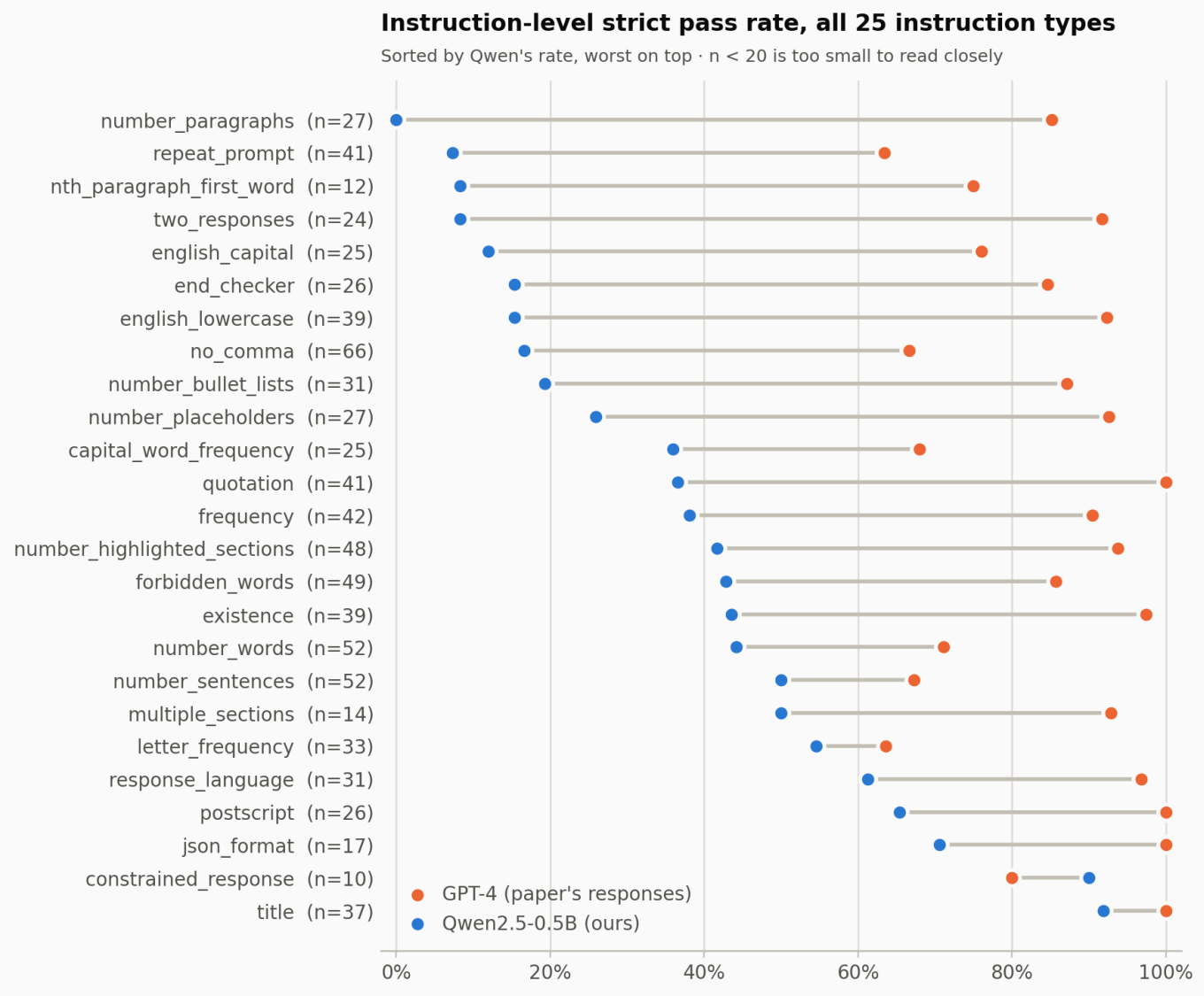
Run directory	Role
`qwen05b_laptop_hf_20260916_004530`	baseline (all figures in Sections 4–6 unless stated)
`qwen05b_laptop_hf_20260916_011950`, `…_013726`, `…_015502`	baseline repeats
`qwen05b_laptop_sys_generic_hf_20260916_021151`	system prompt: generic
`qwen05b_laptop_sys_strict_hf_20260916_022924`	system prompt: follow instructions
`qwen05b_laptop_sys_casual_hf_20260916_024446`	system prompt: conversational
`qwen05b_hf_20260916_003724`	failed attempt, batch 8, out of memory (kept as a record)

Each directory contains:

File	Contents
`config.yaml`, `environment.txt`, `run_manifest.json`	every setting, library version and the resolved model commit
`sample_prompt.txt`, `prompts_rendered.jsonl`	exact strings sent to the model
`generations.jsonl`	all 541 raw responses, verbatim
`eval_results_strict.jsonl`, `eval_results_loose.jsonl`	official per-item scores
`report.json`	all numbers in this document

Appendix: all 25 instruction types (instruction-level strict)

Instruction type	Qwen 0.5B	Passed / n	GPT-4
`length_constraints:number_paragraphs`	0.0%	0 / 27	85.2%
`combination:repeat_prompt`	7.3%	3 / 41	63.4%
`combination:two_responses`	8.3%	2 / 24	91.7%
`length_constraints:nth_paragraph_first_word`	8.3%	1 / 12	75.0%
`change_case:english_capital`	12.0%	3 / 25	76.0%
`change_case:english_lowercase`	15.4%	6 / 39	92.3%
`startend:end_checker`	15.4%	4 / 26	84.6%
`punctuation:no_comma`	16.7%	11 / 66	66.7%
`detectable_format:number_bullet_lists`	19.4%	6 / 31	87.1%
`detectable_content:number_placeholders`	25.9%	7 / 27	92.6%
`change_case:capital_word_frequency`	36.0%	9 / 25	68.0%
`startend:quotation`	36.6%	15 / 41	100.0%
`keywords:frequency`	38.1%	16 / 42	90.5%
`detectable_format:number_highlighted_sections`	41.7%	20 / 48	93.8%
`keywords:forbidden_words`	42.9%	21 / 49	85.7%
`keywords:existence`	43.6%	17 / 39	97.4%
`length_constraints:number_words`	44.2%	23 / 52	71.2%
`detectable_format:multiple_sections`	50.0%	7 / 14	92.9%
`length_constraints:number_sentences`	50.0%	26 / 52	67.3%
`keywords:letter_frequency` †	54.5%	18 / 33	60.6%
`language:response_language`	61.3%	19 / 31	96.8%
`detectable_content:postscript`	65.4%	17 / 26	100.0%
`detectable_format:json_format`	70.6%	12 / 17	100.0%
`detectable_format:constrained_response`	90.0%	9 / 10	80.0%
`detectable_format:title`	91.9%	34 / 37	100.0%



† Contains the two randomly-scored items described in Section 3. Small-n types (n < 20) should not be over-interpreted.

References

- Zhou et al. (2023). *Instruction-Following Evaluation for Large Language Models.* arXiv:2311.07911
- Qwen Team (2024). *Qwen2.5 Technical Report.* arXiv:2412.15115